This is possible using a couple of the advanced Arduino functions for generating randomness. At least these functions generate something of a form of semi-randomness, which is kind of, sort of randomness. Let's now look at how we can use the random functions in our sketches as a form of timing and for some other uses, as well.

random()

The `random()` function returns a semi-random number up to the parameters specified. If no parameters are specified, it will return a value in the signed long data type, with a range of -2,147,483,648 to 2,147,483,647. Its syntax follows:

```
random(min, max)
```

The first parameter, if expressed, is the minimum possible value expected from the `random()` function. This value will be included in the possible outcomes. Likewise, the second value would be the maximum value expected. This value, however, will be excluded from the possible outcomes. So, to generate a random number with 10 possible values you could use the following statement:

```
int randomNumber = random(0, 10);
```

The possible values will include the integers 0 through 9. If this were all we wanted to do, it would be simpler to only specify the maximum value like the following:

```
int randomNumber = random(10);
```

This last statement is functionally the same as the previous one. It is even possible to receive negative random values, although if no minimum has been specified, the `random()` function will assume a value of 0 for its minimum value. As an example of creating a random delay, Listing 7-2 takes our blink without delay example and adds a single line to create some unpredictability.

Listing 7-2. *Random Blink Without Delay*

```
const int ledPin = 13;

int state = LOW;

unsigned long startTime = 0;
unsigned long interval = 500;

void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  if (startTime + interval < millis()) {
    state = !state;
    digitalWrite(ledPin, state);
    startTime = millis();
    interval = random(250, 1001);
  }
}
```